

Atividade 1- eletrônica de Potência

Relatório referente a apresentação da atividade realizada no dia 11/11, o objetivo desse código era apresentar o funcionamento do circuito **RLC** com uma condição inicial em que o capacitor está carregado (900 V) e o circuito começa a evoluir no tempo, neste relatório será apresentado o passo a passo e a descrição do código.

1º passo foi a importação das bibliotecas NumPy, utilizada para operações matemáticas e criação de vetores que armazenam os valores simulados do circuito e PyPlot do Matplotlib, responsável pela geração dos gráficos de corrente e tensão ao final da simulação.

```
main.py > ...  
1  import numpy as np  
2  import matplotlib.pyplot as plt  
3
```

2º passo foi a determinação do dados e das condições iniciais do circuito que foram definidas no enunciado da questão.

```
4  # Dados do circuito  
5  R = 0.085          # ohm  
6  L = 11e-6          # H  
7  C = 180e-6         # F  
8  Vx = 900           # V  
9
```

```
10 # Condições iniciais  
11 i = 0.0  
12 vC = Vx  
13
```

3º passo foi determinado o tempo de simulação:

```
14 # Tempo de simulação  
15 t_max = 400e-6  
16 dt = 0.02e-6  
17 N = int(np.ceil(t_max / dt)) + 1  
18
```

t_max = 400e-6: define o tempo total da simulação, ou seja, até onde o comportamento do circuito será calculado.

dt = 0.02e-6: define o passo de tempo da simulação, determinando a resolução temporal dos cálculos do método RK4.

N = int(np.ceil(t_max / dt)) + 1: calcula quantos passos serão necessários para percorrer todo o intervalo.

Resumidamente Essa linha calcula qual será o tamanho dos vetores de tempo, corrente e tensão:

- **t_max / dt** determina quantos passos são necessários:
 $0,02\mu\text{s}/400\mu\text{s}=20000$
Ou seja, serão necessários 20 000 passos *para chegar até* 400 μs .
- **np.ceil()**: é usado para arredondar o resultado para cima, evitando erros de arredondamento em ponto flutuante que poderiam reduzir indevidamente o número de passos por exemplo se $t_max / dt = 19999.9999996$ e esse valor fosse convertido diretamente para inteiro, ficaria $\text{int}(19999.9999996) = 19999$ e você teria menos um ponto, terminando a simulação ANTES de 400 μs . Portanto **np.ceil()** arredonda para cima, garantindo que nunca falte um ponto: **np.ceil(19999.9999996) = 20000**
- Por fim, soma-se 1 para incluir o instante inicial da simulação ($t = 0$). O valor resultante é o tamanho correto dos vetores de tempo, corrente e tensão utilizados no método de integração RK4.

4º passo é a determinação dos vetores que armazenarão os resultados da simulação:

```
19  # Vetores
20  t = np.zeros(N)
21  i_ind = np.zeros(N)      # Corrente no indutor
22  v_cap = np.zeros(N)      # Tensão no capacitor
23
```

Essas três linhas criam vetores com tamanho N, previamente calculado, todos preenchidos inicialmente com zeros e esses vetores servirão para registrar, em cada passo do método RK4, o instante de tempo, a corrente no indutor e a tensão no capacitor.

```
24  t[0], i_ind[0], v_cap[0] = 0.0, i, vC
25
```

Aqui, o primeiro elemento de cada vetor recebe os valores iniciais do sistema.

5º passo é determinar a função `derivs(i_val, v_val)` calcula as derivadas da corrente e da tensão no circuito RLC, ou seja, como a corrente e tensão estão variando naquele instante.

```
26  # Função derivadas
27  def derivs(i_val, v_val):
28      if v_val > 0:
29          di_dt = v_val / L
30          dv_dt = - i_val / C
31      else:
32          di_dt = (v_val - R * i_val) / L
33          dv_dt = - (i_val + v_val / R) / C
34      return di_dt, dv_dt
35
```

- if $v_val > 0$: determina o momento em que o diodo está DESLIGADO então o circuito funciona como **C e L em série**, sem resistência.
- else:
 $di_dt = (v_val - R * i_val) / L$
 $dv_dt = - (i_val + v_val / R) / C$
assume-se que o diodo conduz e agora aparece o resistor R no circuito, dessa forma a corrente e tensão variam considerando a influência desse resistor.

Essas derivadas são essenciais porque o método RK4 usa exatamente essas taxas de variação para prever o próximo ponto da simulação.

6º passo a implementação do método de Runge–Kutta de 4ª ordem (RK4)

```
36  # Integração RK4
37  for k in range(N - 1):
38      di1, dv1 = derivs(i, vC)
39      di2, dv2 = derivs(i + 0.5 * di1 * dt, vC + 0.5 * dv1 * dt)
40      di3, dv3 = derivs(i + 0.5 * di2 * dt, vC + 0.5 * dv2 * dt)
41      di4, dv4 = derivs(i + di3 * dt, vC + dv3 * dt)
42      i += (dt / 6) * (di1 + 2 * di2 + 2 * di3 + di4)
43      vC += (dt / 6) * (dv1 + 2 * dv2 + 2 * dv3 + dv4)
44
```

O objetivo do RK4 é calcular o próximo valor de corrente e tensão usando uma média ponderada de quatro estimativas de inclinação.

O método **RK4** é uma técnica usada para prever como um sistema evolui ao longo do tempo. Em vez de calcular a próxima etapa usando apenas uma única inclinação (como o método de Euler faz), o RK4 calcula quatro inclinações

diferentes, cada uma estimando o comportamento do sistema em um ponto ligeiramente avançado no tempo. Depois, ele combina essas quatro inclinações em uma média ponderada mais precisa. Assim, cada passo do cálculo considera não só o ponto atual, mas também previsões intermediárias. A operação é dada por:

$$N_{i+1} = N_i + \frac{h}{6} (K_1 + 2K_2 + 2K_3 + K_4),$$

$$K_1 = f(t_i, N_i), \quad K_2 = f\left(t_i + \frac{h}{2}, N_i + \frac{K_1 h}{2}\right),$$

$$K_3 = f\left(t_i + \frac{h}{2}, N_i + \frac{K_2 h}{2}\right) \quad \text{e} \quad K_4 = f(t_i + h, N_i + K_3 h).$$

Adaptando para nossas variáveis i e vC iniciamos com **for k in range(N - 1)**, o código vai repetindo o cálculo **para cada ponto do tempo**, até chegar ao final da simulação.

Em seguida fazemos os cálculos das inclinações:

K1: derivadas no início do intervalo:

- **di1, dv1 = derivs(i, vC):** Calcula as derivadas di/dt e dv/dt usando os valores atuais.

K2: derivadas no meio do intervalo usando k1

- **di2, dv2 = derivs(i + 0.5 * di1 * dt, vC + 0.5 * dv1 * dt):** avançamos meio passo usando k1 e calculamos as derivadas nesse ponto intermediário

K3: segundo cálculo no meio do intervalo usando K2; é mais uma etapa de refinamento da curva.

- **di3, dv3 = derivs(i + 0.5 * di2 * dt, vC + 0.5 * dv2 * dt)**

K4: derivadas no final do intervalo

- **di4, dv4 = derivs(i + di3 * dt, vC + dv3 * dt)**

Depois de calcular k1, k2, k3 e k4, o RK4 combina essas quatro inclinações:

$$i += (dt / 6) * (di1 + 2 * di2 + 2 * di3 + di4)$$

$$vC += (dt / 6) * (dv1 + 2 * dv2 + 2 * dv3 + dv4)$$

Esta fórmula dá mais peso às estimativas centrais (k2 e k3), e menos peso às extremas (k1 e k4).

Importante ressaltar que os valores di_1 , di_2 , di_3 , di_4 (assim como dv_1 , dv_2 , dv_3 e dv_4) representam **derivadas instantâneas** da corrente e da tensão, calculadas em diferentes pontos dentro do intervalo de integração. Esses valores correspondem às inclinações da solução no início, no meio e no final do passo de tempo.

Após calcular essas derivadas, o método combina os quatro valores por meio da expressão:

$$i(t + \Delta t) = i(t) + \frac{\Delta t}{6} (di_1 + 2 di_2 + 2 di_3 + di_4),$$

que constitui uma **aproximação numérica da integral** da derivada $\frac{di}{dt}$ ao longo do intervalo $[t, t + \Delta t]$.

Portanto, no RK4, os termos di_1 – di_4 são **derivadas**, enquanto os valores atualizados i e v_C correspondem a **aproximações da integral** dessas derivadas, resultando na evolução temporal da corrente no indutor e da tensão no capacitor.

7º passo: atualização do vetor tempo e armazenamento dos resultados

```
45     t[k + 1] = t[k] + dt
46     i_ind[k + 1], v_cap[k + 1] = i, vC
47
```

Aqui, cada novo ponto calculado é guardado nos vetores assim formamos a curva completa de corrente e tensão.

8º passo: Após a simulação numérica, foi realizada a identificação de três eventos característicos da resposta temporal do circuito: o instante em que a tensão no capacitor cruza zero, o valor mínimo atingido por essa tensão e o instante em que a corrente no indutor se anula. Esses eventos são detectados a partir da análise dos vetores resultantes da simulação v_cap e i_ind .

```
48     # 1 ponto onde Vc cruza 0 (início da condução do diodo)
49     idx_zero = np.where(np.diff(np.sign(v_cap)))[0]
50     t_zero = t[idx_zero[0]] if len(idx_zero) > 0 else np.nan
51
```

O vetor v_cap é submetido à função $np.sign$, que retorna o sinal de cada amostra (+1, 0 ou -1). Em seguida, $np.diff$ calcula a diferença entre sinais consecutivos. Uma mudança de sinal indica que a tensão passou de positiva para negativa ou

vice-versa, caracterizando um cruzamento por zero. A função `np.where` localiza esse ponto, e o instante correspondente é obtido diretamente do vetor de tempo `t`. Esse evento marca o **início da condução do diodo**.

```
52 # 2 ponto de tensão mínima
53 idx_vmin = np.argmin(v_cap)
54 t_vmin, v_min = t[idx_vmin], v_cap[idx_vmin]
55
```

A função `np.argmin` retorna o índice em que `v_cap` atinge seu menor valor. Esse índice é utilizado para extrair tanto o instante (`t_vmin`) quanto o valor da tensão mínima (`v_min`). Este ponto corresponde ao **maior valor negativo** alcançado pela tensão durante a oscilação, indicando o extremo inferior da dinâmica do capacitor.

```
56 # 3 ponto onde corrente cruza zero (fim da descarga)
57 idx_izero = np.where(np.diff(np.sign(i_ind)))[0]
58 t_izero = t[idx_izero[0]] if len(idx_izero) > 0 else np.nan
59
```

O mesmo procedimento utilizado para detectar o cruzamento de tensão é aplicado ao vetor de corrente. A mudança de sinal em `i_ind` indica o instante em que a corrente se anula, ou seja, quando a energia anteriormente armazenada no campo magnético do indutor se esgota. Esse ponto marca o **fim da descarga do circuito**.

```
60 # Impressões
61 print(f"Tensão cruza 0V em: {t_zero*1e6:.2f} μs")
62 print(f"Tensão mínima: {v_min:.2f} V em {t_vmin*1e6:.2f} μs")
63
```

Essas linhas apresentam ao usuário os principais resultados obtidos a partir da simulação. Os valores são formatados com duas casas decimais e convertidos para microssegundos.

9º passo: plotagem dos gráficos

A etapa de plotagem gera dois gráficos: o primeiro mostra a corrente no indutor ao longo do tempo, destacando com linhas verticais os instantes em que a tensão do capacitor cruza zero, atinge o valor mínimo e quando a corrente se anula. O segundo gráfico exibe a evolução da tensão no capacitor, usando as mesmas

marcações temporais para facilitar a comparação entre as duas grandezas. Cada gráfico possui título, eixos identificados, grade e legenda, permitindo visualizar claramente os principais eventos do processo dinâmico do circuito.

```
64 # Plot – Corrente no indutor
65 plt.figure(figsize=(11, 7))
66 plt.subplot(2, 1, 1)
67 plt.plot(t * 1e6, i_ind, 'b', linewidth=2)
68 plt.axvline(t_zero * 1e6, color='gray', linestyle='--', label='Vc = 0 V')
69 plt.axvline(t_vmin * 1e6, color='red', linestyle='--', label='Vc mín (≈ -200 V)')
70 plt.axvline(t_izero * 1e6, color='green', linestyle='--', label='iL = 0 A')
71 plt.title('Corrente no Indutor i_ind(t)')
72 plt.ylabel('Corrente (A)')
73 plt.grid(True)
74 plt.legend()
75
76 # Plot – Tensão no capacitor
77 plt.subplot(2, 1, 2)
78 plt.plot(t * 1e6, v_cap, 'r', linewidth=2)
79 plt.axvline(t_zero * 1e6, color='gray', linestyle='--', label='Vc = 0 V')
80 plt.axvline(t_vmin * 1e6, color='red', linestyle='--', label='Vc mín (≈ -200 V)')
81 plt.axvline(t_izero * 1e6, color='green', linestyle='--', label='iL = 0 A')
82 plt.title('Tensão no Capacitor v_cap(t)')
83 plt.xlabel('Tempo (μs)')
84 plt.ylabel('Tensão (V)')
85 plt.grid(True)
86 plt.legend()
87
88 plt.tight_layout()
89 plt.show()
```